

18

Cyber Security
Codec Technologies India
Internship 2026

Secure File Sharing Platform

End-to-end encryption; a blind server

AT A GLANCE

AES-256-GCM

Encryption

RSA-OAEP / scrypt

Key wrap

False

Plaintext on disk

16

Audit entries

15 / 15

Recorded tests

no (argon2)

Re-run here

ciphertext only

Server sees

VERDICT

TESTS 15/15

RECORDED RUN

DEPENDENCY UNAVAILABLE

DEFENSIVE ONLY

SYNTHETIC

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Objective & brief mapping

The brief asked for a secure file-sharing platform. I built an end-to-end-encrypted one where the server is cryptographically blind: it stores only ciphertext blobs and wrapped (encrypted) data keys, never plaintext and never the raw keys. On top I added accounts (Argon2), owner-based access control, expiring share links (time and max-download), link revocation, an append-only audit log, and hardened headers.

Architecture

Envelope encryption: the client generates a random 256-bit data key, encrypts the file with AES-256-GCM under it, then wraps the data key with either the recipient's RSA-OAEP public key or a scrypt-derived passphrase key. Only the ciphertext blob and wrapped key are uploaded; decryption happens entirely on the recipient's side.

```
data_key=random(32B); ct=AESGCM(data_key).enc(file)
wrapped=RSA_enc(bob_pub,data_key) -> server stores {ct, wrapped}
Bob: data_key=RSA_dec(bob_priv,wrapped); file=AESGCM.dec(ct)
```



Rendered platform dashboard (results/dashboard_screenshot.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python demo.py # full Alice->Bob story + refused links
python run.py # http://127.0.0.1:5018
pytest -q # recorded: 15 passed (argon2 unavailable here)
```

The artefacts each figure and table above are drawn from:

- **demo_transcript.txt** — end-to-end flow
- **audit_log.csv** — 16 append-only entries
- **dashboard_screenshot.png** — Playwright render
- **index_render.html** — rendered UI

Method & implementation

The crypto module is a clean importable library (keygen, encrypt_file, decrypt_file, wrap/unwrap with RSA or passphrase). GCM's auth tag means any tampering fails decryption. Share links expire by time and by max_downloads and can be revoked; every action appends to an audit table exported to CSV.

Results

From results/demo_transcript.txt and audit_log.csv. The demo asserts every control and scans on-disk blobs for plaintext markers.

Control / check	Result
RSA-wrapped Alice->Bob share	round-trip OK, 70-byte secret exact
max_downloads=1	1st download 200, 2nd 410
Expired / revoked link	410 / 410
Plaintext on disk (3 blobs)	present = False
Audit export	16 entries -> CSV

Envelope encryption steps

Step	Operation	On
1	data_key = random(32B)	client
2	AES-256-GCM encrypt file	client
3	wrap key (RSA-OAEP / scrypt)	client
4	upload ct + wrapped only	server (blind)

Verification

KEY TAKEAWAY Envelope encryption where the server stores only ciphertext and wrapped keys — the demo scanned every on-disk blob and confirmed plaintext present = False.

18

Cyber Security
Codec Technologies India
Internship 2026

Secure File Sharing Platform

End-to-end encryption; a blind server

AT A GLANCE

AES-256-GCM

Encryption

RSA-OAEP / scrypt

Key wrap

False

Plaintext on disk

16

Audit entries

15 / 15

Recorded tests

no (argon2)

Re-run here

ciphertext only

Server sees

VERDICT

TESTS 15/15

RECORDED RUN

DEPENDENCY UNAVAILABLE

DEFENSIVE ONLY

SYNTHETIC

Eswar Mahalingam

Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Tests not re-run here — argon2-cffi is unavailable in this container and the app imports it at load. The recorded run is **15 passed** (~4.8 s). I verified the demo transcript, audit CSV and dashboard screenshot artefacts are present and consistent with the README.

Test group	Checks	Result (recorded)
Crypto	RSA + passphrase round-trips	PASS
Negative	wrong passphrase/key, tamper -> fail	PASS
Server blindness	on-disk blob is ciphertext	PASS
Links/CSRF/headers	expired/revoked refused, CSRF, headers	PASS
Recorded total	15 tests	15/15

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

The crypto core is real; the one demo shortcut (server-held private key) is named as the biggest gap.

Ran here (recorded)	Real E2E	Note
server stores private key	client-held key (WebCrypto)	biggest demo-vs-product gap
Argon2id hashing	argon2-cffi	unavailable in this container
local disk blobs	IPFS pinning (CID)	design is pluggable

Ethics, data integrity & declared substitutions

Educational and defensive. All users (alice/bob) and files are synthetic and project-generated; no real credentials or documents. **Declared substitution / honest gap (copied from my README):** the demo stores each user's RSA private key in the server DB purely so it is self-contained and runnable — a true E2E system must keep the private key on the client (e.g. WebCrypto non-extractable keys) and upload only the public key. This is the single biggest gap between the demo and a real product, and I state it plainly. The bundled Flask server is dev-only.

Learnings

Envelope encryption finally clicked when I saw the server logs contain nothing but opaque blobs and wrapped keys. Being explicit about the client-key gap taught me that honest limitation-writing is part of a security deliverable: a demo that hides its weakest assumption is worse than one that names it.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- The demo stores each RSA private key in the server DB so it is self-contained — a true E2E keeps it client-side.
- The bundled Flask server is dev-only; production needs a WSGI server behind TLS.
- Login rate-limiting is not implemented here (it is in projects 15/20).
- No forward secrecy or key rotation; a leaked private key exposes past shares.
- Envelope encryption: AES-256-GCM per file, data key wrapped with RSA-OAEP or scrypt.
- Server stores only ciphertext + wrapped key; plaintext-on-disk scan returned False.
- Share links expire by time and by max_downloads, and can be revoked (HTTP 410).
- GCM auth tag makes any ciphertext tampering fail decryption.
- RSA-wrapped Alice->Bob share round-trips a 70-byte secret exactly.
- max_downloads=1: first download 200, second 410.
- Expired and revoked links both return 410.
- On-disk blob scan: plaintext present = False.
- 16 audit entries exported to CSV.
- GCM auth tag makes any ciphertext tamper fail decryption.
- IPFS pinning is a documented, pluggable storage option.
- References: NIST SP 800-38D, RFC 8017/7914, Argon2, OWASP.
- Reference: NIST SP 800-38D (AES-GCM); RFC 8017 (RSA-OAEP).